



CLASS FUNDAMENTALS

Mr. Sagar Patel

What is Class?

“A class is the template or blueprint from which objects are made.”

- It represents the **set of properties or methods** that are **common to all objects of one type**.

What is Class? Cont..

- In general, class declarations can include these components, in order:
- Modifiers: A class can be public or have default access.
- Class name: The name should begin with an initial letter (capitalized by convention).
- Superclass(if any): The name of the class's parent (superclass), if any, preceded by the keyword extends. A class can only extend (subclass) one parent.
- Body: The class body surrounded by braces, { }.

What is Class? Cont..

Identity

Name of dog

State/Attributes

Breed

Age

Color

Behaviors

Bark

Sleep

Eat

Simple Program in Java

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

How to Run:

1. Save the file as `HelloWorld.java`
2. Open terminal/command prompt and navigate to the file directory.
3. Compile:

```
bash
```

```
javac HelloWorld.java
```

4. Run:

```
bash
```

```
java HelloWorld
```

Output:

```
Hello, World!
```

Valid main method signatures

```
public static void main(String[] args) // ✓ Standard form
public static void main(String []args) // ✓ Brackets after type or variable – both allowed
public static void main(String args[]) // ✓ Array notation on variable – also valid
public static void main(String... args) // ✓ Varargs – allowed as main method parameter
static public void main(String[] args) // ✓ Modifiers can be in any order
public static final void main(String[] a) // ✓ 'final' allowed
final public static void main(String[] a) // ✓ Modifier order flexible
final strictfp public static void main(String[] a) // ✓ 'strictfp' also allowed
```

Simple Program in Java

- Step 1: Class Declaration

```
public class Student {
```

- Step 2: Private Data Members

```
private int stdIdNo;
```

```
private String stdName;
```

- Step 3: Default Constructor

```
public Student() {
```

```
    System.out.println("Constructor  
is initiated...");  
}
```

- Step 4: Getter Method for **stdIdNo**

```
public int getStdIdNo() {  
    return stdIdNo;  
}
```

```
}
```

- Step 5: Setter Method for **stdIdNo**

```
public void setStdIdNo(int stdIdNo) {  
    this.stdIdNo = stdIdNo;  
}
```

- Step 6: Getter Method for **stdName**

```
public String getStdName() {  
    return stdName;  
}
```

- Step 7: Setter Method for **stdName**

```
public void setStdName(String stdName) {  
    this.stdName = stdName;  
}
```

Full Program Recap

```
public class Student {  
  
    private int stdIdNo; // Student ID number (private)  
    private String stdName; // Student name (private)  
  
    // Constructor  
    public Student(){  
        System.out.println(" Constructor is initiated...");  
    }  
  
    // Getter for ID  
    public int getStdIdNo() {  
        return stdIdNo;  
    }  
  
    // Setter for ID  
    public void setStdIdNo(int stdIdNo) {  
        this.stdIdNo = stdIdNo;  
    }  
  
    // Getter for Name  
    public String getStdName() {  
        return stdName;  
    }  
  
    // Setter for Name  
    public void setStdName(String stdName) {  
        this.stdName = stdName;  
    }  
}
```

Simple Program in Java

- Step 1: Import Required Classes

```
import java.util.Scanner;
```

- Step 2: Define the **Admin** Class

```
public class Admin {
```

- Step 3: Define the **main()** Method

```
public static void main(String[] args) {
```

- Step 4: Create Object of Student Class

```
Student s = new Student();
```

- Step 5: Create Scanner Object

```
Scanner scanf = new Scanner(System.in);
```

- Step 6: Get Student ID from User

```
System.out.println("Enter Student ID");
```

```
int stdId = scanf.nextInt();
```

- Step 7: Get Student Name from User

```
System.out.println("Enter Student Name");
```

```
String stdName = scanf.next();
```

- Step 8: Set and Display Student ID

```
s.setStdIdNo(stdId);
```

```
System.out.println("Student ID is: " +  
s.getStdIdNo());
```

- Step 9: Set and Display Student Name

```
s.setStdName(stdName);
```

```
System.out.println("Student Name is: " +  
s.getStdName());
```

Full Program Recap

```
// Admin class with main method
public class Admin {
    public static void main(String[] args) {

        Student s = new Student(); // Create object of Student
        Scanner scanf = new Scanner(System.in); // Create Scanner object

        System.out.println("Enter Student ID:");
        int stdId = scanf.nextInt(); // Input ID

        scanf.nextLine(); // To consume leftover newline

        System.out.println("Enter Student Name:");
        String stdName = scanf.nextLine(); // Input Name (allows spaces)

        s.setStdIdNo(stdId); // Set ID
        s.setStdName(stdName); // Set Name

        // Display stored data
        System.out.println("Student ID is: " + s.getStdIdNo());
        System.out.println("Student Name is: " + s.getStdName());

        scanf.close(); // Close Scanner
    }
}
```

What is a Constructor?

- Constructors are used to initialize the object's state.
- Like **methods**, a constructor also contains a **collection of statements** (i.e., instructions) that are executed at the **time of Object creation**.

When is a Constructor called?

- Each time an object is created using `new()` keyword at least one constructor (it could be the default constructor) is invoked to assign initial values to the data members of the same class.

- Example:

```
class Student{
```

```
.....
```

```
// A Constructor
```

```
public Student() {}
```

```
.....
```

```
}
```

```
// We can create an object of Student class  
// using the below statement. This statement  
// calls above constructor.
```

```
Student subj = new Student();
```

Rules for writing Constructor

- Constructor(s) of a class **must has same name as the class name** in which it resides.
- A constructor in Java **can not be abstract, final, static and Synchronized**.
- **Access modifiers can be used** in constructor declaration to control its access i.e which other class can call the constructor.

Types of Constructors

- There are two types of constructors in Java:
 1. **No-argument constructor**: A constructor that has no parameter is known as default constructor.
- Note:
 - If we don't define a constructor in a class, then compiler creates default constructor(with no arguments) for the class.
 - And if we write a constructor with arguments or no-argument then compiler does not create default constructor.
 - Default constructor provides the default values to the object like 0, null etc. depending on the type.

```
// Student.java
public class Student {
    String name;
    int id;

    public Student() {
        System.out.println("Constructor called");
    }
}
```

```
// Admin.java
public class Admin {
    public static void main(String[] args) {
        Student s = new Student();
        System.out.println("Student Name: " + s.name); // will print null
        System.out.println("Student ID: " + s.id);      // will print 0
    }
}
```

```
Constructor called
Student Name: null
Student ID: 0
```

Types of Constructors

2. **Parameterized Constructor:** A constructor that has parameters is known as parameterized constructor.

 - If we want to initialize fields of the class with our own values, then use parameterized constructor.

```
class Student {  
    String name;  
    int id;  
  
    public Student(String sname, int sid) {  
        name = sname;  
        id = sid;  
    }  
}  
  
public class Admin {  
    public static void main(String[] args) {  
        Student s = new Student("Nikunj", 101);  
        System.out.println("Student Name: " + s.name + " and Student ID: " + s.id);  
    }  
}
```

Student Name: Nikunj and Student ID: 101

Does constructor return any value?

- There are no “return value” statements in constructor, but constructor returns current class instance. We can write ‘return’ inside a constructor.

// Constructor with return; statement

```
public Student(String name, int age) {  
    if (age < 0) {  
        System.out.println("Invalid age. Age must be positive.");  
        return; // exits the constructor early, object still created but fields not set  
    }  
    this.name = name;  
    this.age = age;  
}
```

What is Overloading ?

- Overloading allows different constructor/methods to have same name, but different signatures.
- Where signature can differ by number of input parameters or type of input parameters or both.
- Overloading is related to compile time (or static) polymorphism.

Constructor Overloading

- Compiler differentiates constructors on the basis of numbers of parameters, types of the parameters and order of the parameters.

```
// Student.java
class Student {

    // Constructor with one String argument
    public Student(String name) {
        System.out.println("Constructor with one argument - String : " + name);
    }

    // Constructor with String and int arguments
    public Student(String name, int id) {
        System.out.println("Constructor with two arguments : String and Integer : " + name + " " + id);
    }

    // Constructor with one long argument
    public Student(long CellNo) {
        System.out.println("Constructor with one argument : Long : " + CellNo);
    }
}
```

```
// Admin.java  
public class Admin {  
    public static void main(String[] args) {  
        Student s1 = new Student("Nikunj");  
        Student s2 = new Student("Nikunj", 101);  
        Student s3 = new Student(1234567890L); // Note the 'L' to denote long literal  
    }  
}
```

Constructor with one argument - String : Nikunj

Constructor with two arguments : String and Integer : Nikunj 101

Constructor with one argument : Long : 1234567890

How constructors are different from methods in Java?

- Constructor(s) must have the same name as the class within which it defined while it is not necessary for the method in java.
- Constructor(s) do not have any return type while method(s) have the return type or void (if does not return any value).
- Constructor is called only once at the time of Object creation while method(s) can be called any numbers of time.

Constructor Chaining

- Constructor chaining is the process of calling one constructor from another constructor with respect to current object.
- Constructor chaining can be done in two ways:
- Within same class: It can be done using `this()` keyword for constructors in same class
- From base class: by using `super()` keyword to call constructor from the base class.

'this' keyword in JAVA

- 'this' is a **reference variable** that refers to the **current object**.
1. Using 'this' keyword to refer current class instance variables

```
class Student {  
    String name;  
    int age;  
  
    Student(String name, int age) {  
        this.name = name; // 'this.name' refers to the instance variable  
        this.age = age;    // 'age' is the constructor parameter  
    }  
  
    void display() {  
        System.out.println("Name: " + this.name + ", Age: " + this.age);  
    }  
}
```

'this' keyword in JAVA

2. Using this() to invoke current class constructor

```
public class Temp {  
    int x;  
  
    public Temp() {  
        this(5);  
        System.out.println("The Default constructor");  
    }  
  
    public Temp(int x) {  
        this(5, 15);  
        System.out.println(x);  
    }  
  
    public Temp(int x, int y) {  
        System.out.println(x * y);  
    }  
  
    public static void main(String args[]) {  
        new Temp();  
    }  
}
```

```
public class Student {  
    int id;  
    String name;  
  
    // Regular constructor  
    public Student(int i, String n) {  
        id = i;  
        name = n;  
    }  
  
    // Copy constructor  
    public Student(Student s) {  
        id = s.id;  
        name = s.name;  
    }  
  
    public void display() {  
        System.out.println(id + " " + name);  
    }  
  
    public static void main(String args[]) {  
        Student s1 = new Student(101, "Nikunj"); // original object  
        Student s2 = new Student(s1);           // copied object using copy constructor  
        s1.display(); // Output: 101 Nikunj  
        s2.display(); // Output: 101 Nikunj  
    }  
}
```

“static” keyword in java

- Static is a non-access modifier in Java which is applicable for the following:
 - blocks (static clause)
 - Variables (also known as class variable)
 - Methods (also known as class method)
 - nested classes

Static blocks

```
public class Test {  
    static int a = 10;  
    static int b;  
  
    // static block  
    static {  
        System.out.println("Static block initialized.");  
        b = a * 4;  
    }  
  
    public static void main(String[] args) {  
        System.out.println("from main");  
        System.out.println("Value of a : " + a);  
        System.out.println("Value of b : " + b);  
    }  
}
```

- Static block that gets executed **exactly once**, when the **class is first loaded**.
- Static blocks are **executed before constructors**.

```
Static block initialized.  
from main  
Value of a : 10  
Value of b : 40
```

```
public class Test {  
  
    static int i;  
    int j;  
  
    static {  
        i = 10;  
        System.out.println("static block called");  
    }  
  
    public Test() {  
        System.out.println("Constructor called");  
    }  
}  
  
public class TestMain {  
  
    public static void main(String args[]) {  
  
        Test t1 = new Test();  
        Test t2 = new Test();  
  
    }  
}
```

```
static block called  
Constructor called  
Constructor called
```

Static variables

```
public class Test {  
  
    static int a = m1(); // (1)  
  
    static {  
        System.out.println("Inside static block"); // (2)  
    }  
  
    static int m1() {  
        System.out.println("from m1"); // (1a)  
        return 20;  
    }  
  
    public static void main(String[] args) {  
        System.out.println("Value of a : " + a); // (3)  
        System.out.println("from main"); // (4)  
    }  
}
```

- When a variable is declared as static, then a **single copy of variable** is created and **shared among all objects at class level.**

```
from m1  
Inside static block  
Value of a : 20  
from main
```

Static Method

- Methods declared as static have several restrictions:
 - They can only directly **call other static methods**.
 - They can only directly **access static data**.
 - They **cannot refer** to 'this' or 'super' in any way.

```
public class Test {  
  
    static int a = 10;    // ✓ static variable  
  
    int b = 20;           // ✓ instance (non-static) variable  
  
    static void m1() {  
        a = 20;    // ✓ allowed (static method accessing static variable)  
        System.out.println("from m1");  
  
        b = 10;    // ✗ ERROR: Cannot access non-static variable from static context  
        m2();      // ✗ ERROR: Cannot call non-static method from static context  
        System.out.println(this.a); // ✗ ERROR: Cannot use 'this' in static context  
        System.out.println(super.a); // ✗ ERROR: Cannot use 'super' in static context  
    }  
  
    void m2() {  
        System.out.println("from m2");  
    }  
  
    public static void main(String[] args) {  
        // nothing here yet  
    }  
}
```

Instance initializer block

- Initializer block contains the code that **is always executed** whenever an **instance(object)** is created.

```
public class Test {  
  
    // Initializer block starts..  
    {  
        // This code is executed before every constructor.  
        System.out.println("Common part of constructors invoked !!");  
    }  
    // Initializer block ends  
  
    public Test() {  
        System.out.println("Default Constructor invoked");  
    }  
  
    public Test(int x) {  
        System.out.println("Parametrized constructor invoked");  
    }  
  
    public static void main(String[] arr) {  
        Test obj1, obj2;  
        obj1 = new Test();  
        obj2 = new Test(0);  
    }  
}
```

Common part of constructors invoked !!
Default Constructor invoked
Common part of constructors invoked !!
Parametrized constructor invoked

```
public class InstanceInitializerDemo {  
    private int x;  
    private int y;  
  
    // Instance initializer block  
    {  
        x = 10;  
        y = 20;  
        System.out.println("Instance initializer block executed");  
    }  
  
    public InstanceInitializerDemo() {  
        System.out.println("Constructor with no arguments");  
    }  
  
    public InstanceInitializerDemo(int x) {  
        this.x = x;  
        System.out.println("Constructor with one argument");  
    }  
  
    public static void main(String[] args) {  
        new InstanceInitializerDemo();  
        new InstanceInitializerDemo(30);  
    }  
}
```

```
Instance initializer block executed  
Constructor with no arguments  
Instance initializer block executed  
Constructor with one argument
```

Static vs Instance initializer block

- Order of execution:
 - `static` initialization blocks of `super classes`. (Runs once)
 - `static` initialization blocks of the class. (Runs once)
 - `instance` initialization blocks of `super classes`
 - constructors of `super classes`
 - `instance` initialization blocks of the class
 - constructor of the class.
- Note: The code in `static initialization` block will be `executed at class load time before any instances of the class` are constructed and `before any static` methods are called.

```
public class Test {  
  
    static int staticVariable;  
    int nonStaticVariable;  
  
    // Static block: executes once when the class is loaded  
    static {  
        System.out.println("Static initialization.");  
        staticVariable = 5;  
    }  
  
    // Instance block: executes every time an object is created, before the constructor  
    {  
        System.out.println("Instance initialization.");  
        nonStaticVariable = 7;  
    }  
  
    // Constructor  
    public Test() {  
        System.out.println("Constructor.");  
    }  
  
    public static void main(String[] args) {  
        new Test(); // First object  
        new Test(); // Second object  
    }  
}
```

```
Static initialization.  
Instance initialization.  
Constructor.  
Instance initialization.  
Constructor.
```

final keyword

- final is a non-access modifier applicable only to a variable, a method or a class.
- Following are different contexts where final is used.
 - Final Variable  To Create Constant Variable
 - Final Method  Prevent Method Overriding
 - Final Class  Prevent Inheritance
- Note: We must initialize a final variable, otherwise compiler will throw compile-time error

final Keyword

- There are three ways to initialize a final variable :
 - You can initialize a final variable when it is declared.
 - A blank final variable can be initialized inside an instance-initializer block or inside a constructor.
 - Note: If you have more than one constructor in your class, then it must be initialized in all of them; otherwise, a compile-time error will be thrown.
 - A blank final static variable can be initialized inside a static block.

```
public class FinalEx {  
    private final int a;  
    private final int b;  
    private final int c = 10;  
  
    {  
        a = 10;    // ← Instance initializer assigning `a`  
        b = 20;    // ← Instance initializer assigning `b`  
    }  
  
    public FinalEx(int val) {  
        a = 10;    // ✗ ERROR: `a` is already assigned in initializer block  
    }  
  
    public FinalEx(int val1, int val2){  
        a = 10;    // ✗ ERROR: `a` is already assigned in initializer block  
    }  
}
```

Garbage Collection

- Garbage Collection is process of reclaiming the runtime unused memory automatically. In other words, it is a way to destroy the unused objects.
- An automatic garbage collector essentially performs two tasks:
 - Decides if and when memory needs to be reclaimed
 - Finds objects that are no longer needed by the program and reclaims their storage
- Note: we were using `free()` function in C language and `delete()` in C++. But, in java it is performed automatically or Manually using `System.gc()` method.

How can an object be unreferenced?

- By nulling the reference
Employee e=new Employee();
e=null;
- By assigning a reference to another
Employee e1=new Employee();
Employee e2=new Employee();
e1=e2; //now the first object referred by e1 is eligible for garbage collection
- By an anonymous object
new Employee();

finalize() method

- The finalize() method is **invoked each time before the object is garbage collected**.
- This method can be used to perform cleanup processing. This method is defined in Object class as:

```
protected void finalize(){} 
```

- Note: Instead, it's recommended to use explicit resource management techniques like **try-with-resources** or **try-finally blocks**, or implement the **AutoCloseable** interface for managing resources that need explicit cleanup.

Java Inner Class

- Java inner class or nested class is a class, i.e., declared inside another class or interface.
- We use inner classes to logically group classes and interfaces in one place, making them **more readable and maintainable**.
- Additionally, it can access all the members of the outer class, including private data members and methods.

```
class Java_Outer_class{  
    //code  
    class Java_Inner_class{  
        //code  
    }  
}
```

Advantages of Java inner classes

- There are three advantages of inner classes in Java. They are as follows:
- Nested classes represent a special type of relationship that is it can access all the members (data members and methods) of outer class including private.
- Nested classes are used to develop more readable and maintainable code because it logically group classes and interfaces in one place only.
- **Code Optimization**: It requires less code to write.

Types of Nested classes

- There are two types of nested classes: nonstatic and static nested classes.
- 1. The nonstatic nested classes are also known as inner classes.
 1. Nonstatic nested class(inner class)
 2. Member inner class
 3. Anonymous inner class
 4. Local inner class
- 2. Static nested class

Type	Description
Member Inner Class	A class created within class and outside method.
Anonymous Inner Class	A class created for implementing interface or extending class. Its name is decided by the java compiler.
Local Inner Class	A class created within method.
Static Nested Class	A static class created within class.

Java Member inner class

```
class TestMemberOuter1{
    private int data = 30;
    class Inner{
        void msg(){
            System.out.println("data is " + data);
        }
    }
    public static void main(String args[]){
        TestMemberOuter1 obj = new TestMemberOuter1();
        TestMemberOuter1.Inner in = obj.new Inner();
        in.msg();
    }
}
```

Java Anonymous inner class

```
abstract class Person{
    abstract void eat();
}

class TestAnonymousInner{
    public static void main(String args[]){
        Person p=new Person(){
            void eat(){System.out.println("nice fruits");}
        };
        p.eat();
    }
}
```

Java Local inner class

```
public class localInner1{
    private int data=30;           //instance variable
    void display(){
        class Local{
            void msg(){System.out.println(data);}
        }
        Local l=new Local();
        l.msg();
    }
    public static void main(String args[]){
        localInner1 obj=new localInner1();
        obj.display();
    }
}
```

Java static nested class

- A static class, i.e., one created inside another class, is referred to as a static nested class in Java. It cannot access non-static data members and methods. The outer class can access the static inner class.
 - It can access static data members of the outer class, including private.
 - A static nested class cannot access non-static (instance) data members or methods.

```
class TestOuter1{
    static int data=30;
    static class Inner{
        void msg(){System.out.println("data is "+data);}
    }
    public static void main(String args[]){
        TestOuter1.Inner obj=new TestOuter1.Inner();
        obj.msg();
    }
}
```

Any Questions?